STM32CubeIDE VS Code Setup Guide

for CMOS & DeviceDriver Framework

Table of Contents

1 Audience 3
2 Download & Install 3
3 STM32CubeIDE 4
3.1 Workspace 4
3.2 Project 7
3.2.1 Step 1 7
3.2.2 Step 2 8
3.2.3 Step 3 9
3.2.4 Step 4 10
3.2.5 Step 5 12
3.2.6 Step 6 14
3.2.7 Step 7 20
3.2.8 Step 8 21
4 VS Code 22
4.1 Extensions 22
4.2 Workspace 23
5 Revision History 30

1 Audience

This guide is for unexperienced users that just want to setup their software environment.
More experienced users can also use this guide and tweak the settings a bit if they know what they are doing.

All the steps here are provided in very detail to make it easy for new users to create a working environment.
If questions arise – just contact me on Github.

2 Download & Install

CMOS: https://github.com/leonuetzel/CMOS

DeviceDriver: https://github.com/leonuetzel/DeviceDriver

STM32CubeIDE: https://www.st.com/en/development-tools/stm32cubeide.html#get-software

VS Code: https://code.visualstudio.com/download

Important: This guide is only valid for STM32CubeIDE Versions < 2.0 (I recommend v1.9)

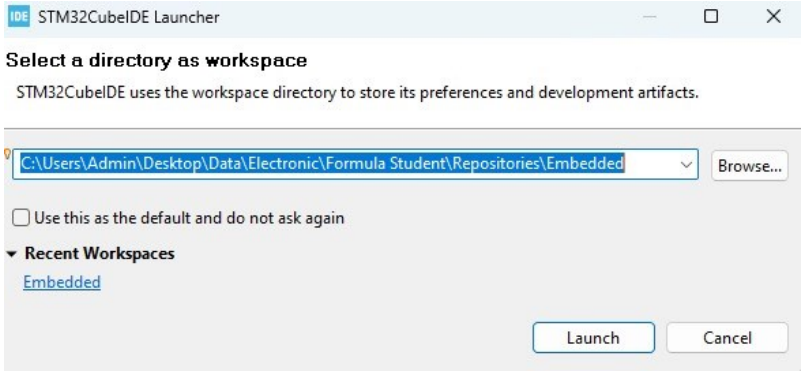
Install the STM32CubeIDE and VS Code using the respective default installer settings.

To download the STM32CubeIDE, you need to log into your ST Account.
If you don't have one, you need to create one.

3 STM32CubeIDE

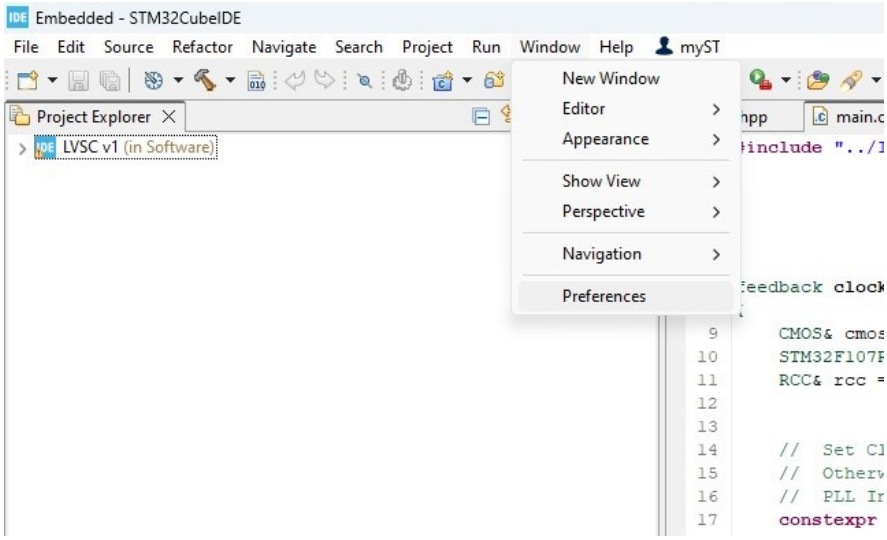
3.1 Workspace

Once installed, simply launch the software and select a workspace.

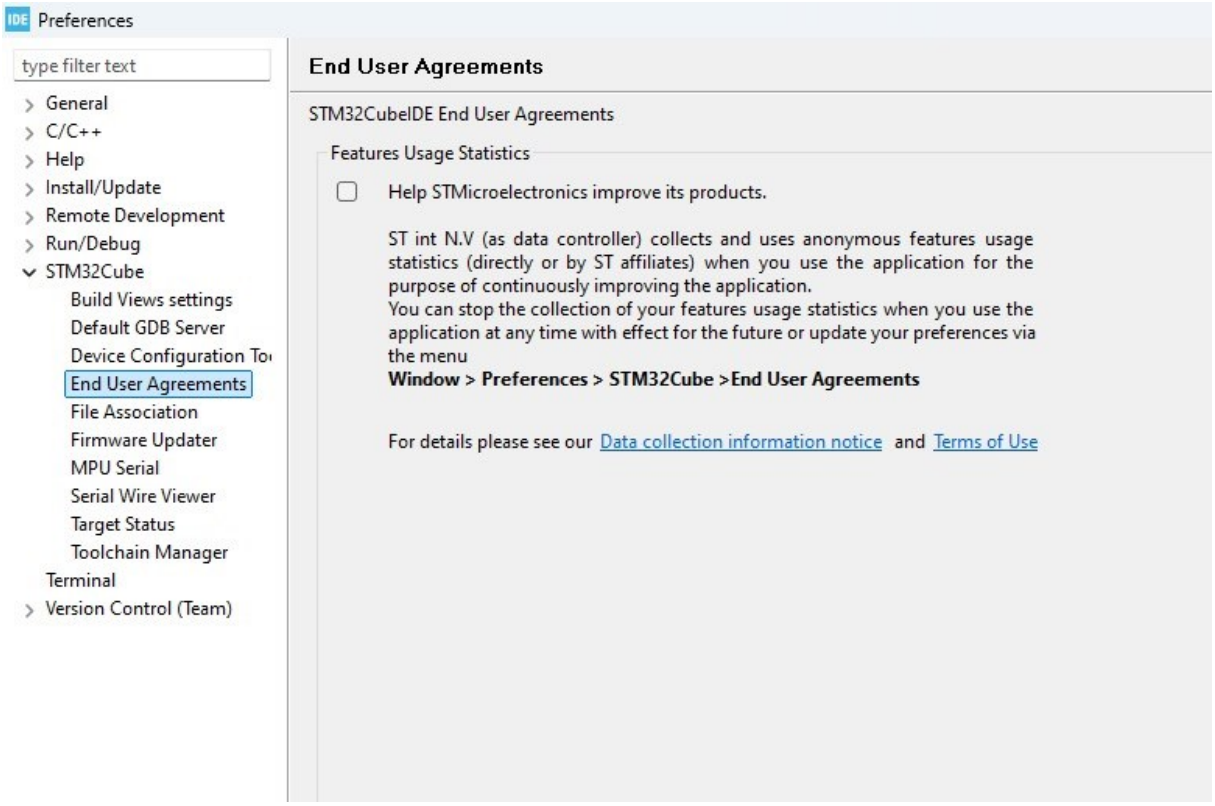


Optional: Disable the data collection by ST.

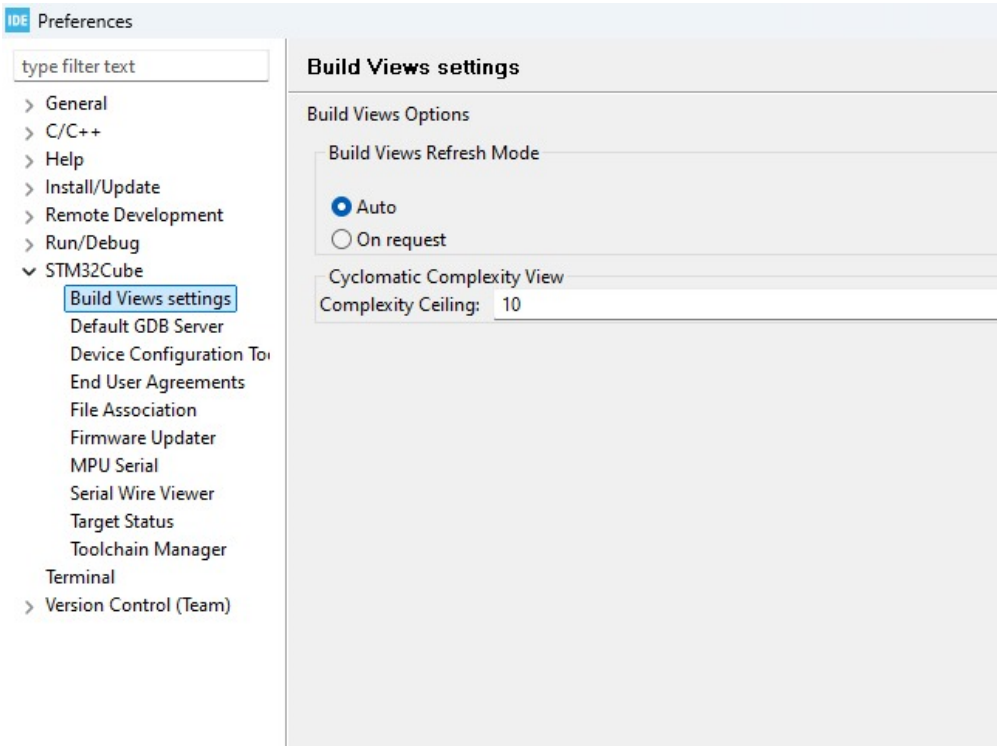
"Window"->"Preferences"



Disable the box "Help STMicroelectronics improve its products".



Now set the build view options mode to "Auto".



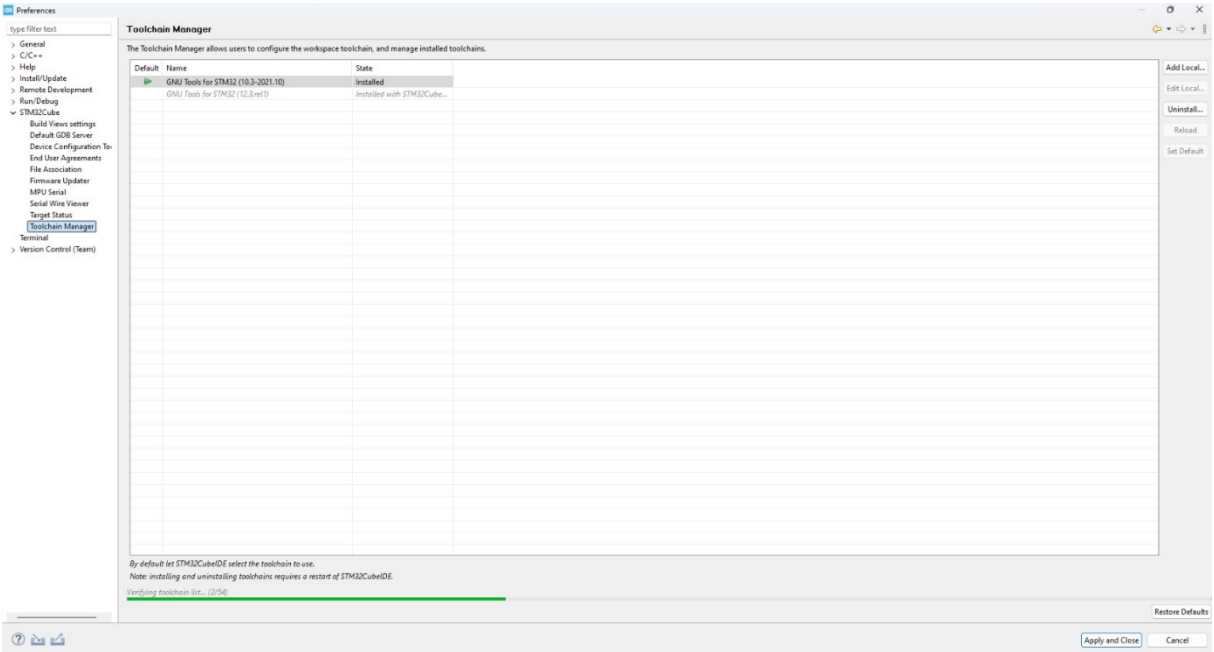
Open the “Toolchain Manager” to install the required Toolchain:

"GNU Tools for STM32 (10.3-2021.10)"

Before it can be selected for installation, the window will load all available toolchains, which takes quite a lot of time (no idea what they are doing there!). Once it has finished loading all available toolchains, select the required one and hit the "Install" button on the right.

There will be some questions for Permission – you need to trust them in order to continue - and go for it.

If the IDE asks for a restart after installation - restart it.



Once installed, open the “Toolchain Manager” again, select the toolchain by clicking one time on it and hit the button "Set Default" on the right to make it our default toolchain.

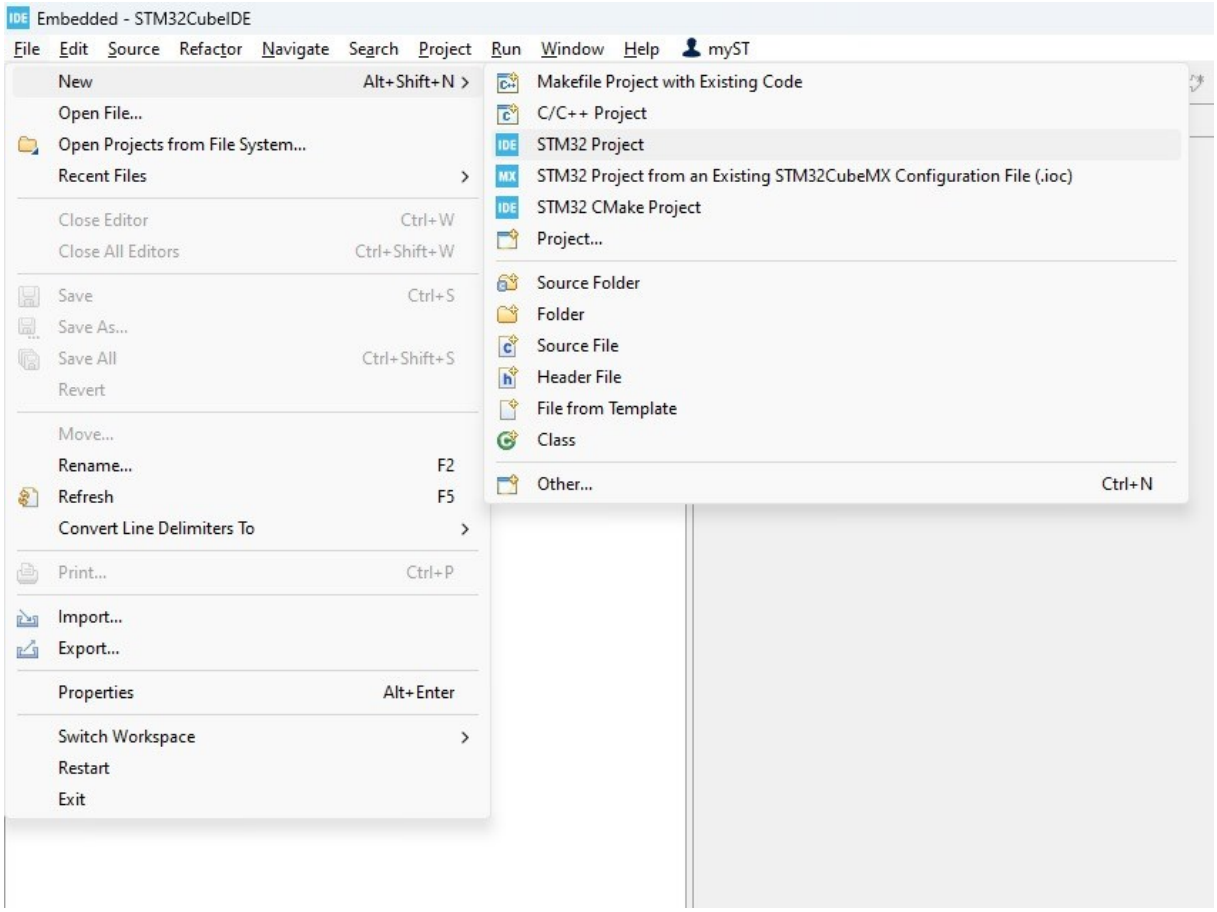
Now we are ready to set up a project :)

3.2 Project

Setting up a new project isn't complicated, but takes many steps...

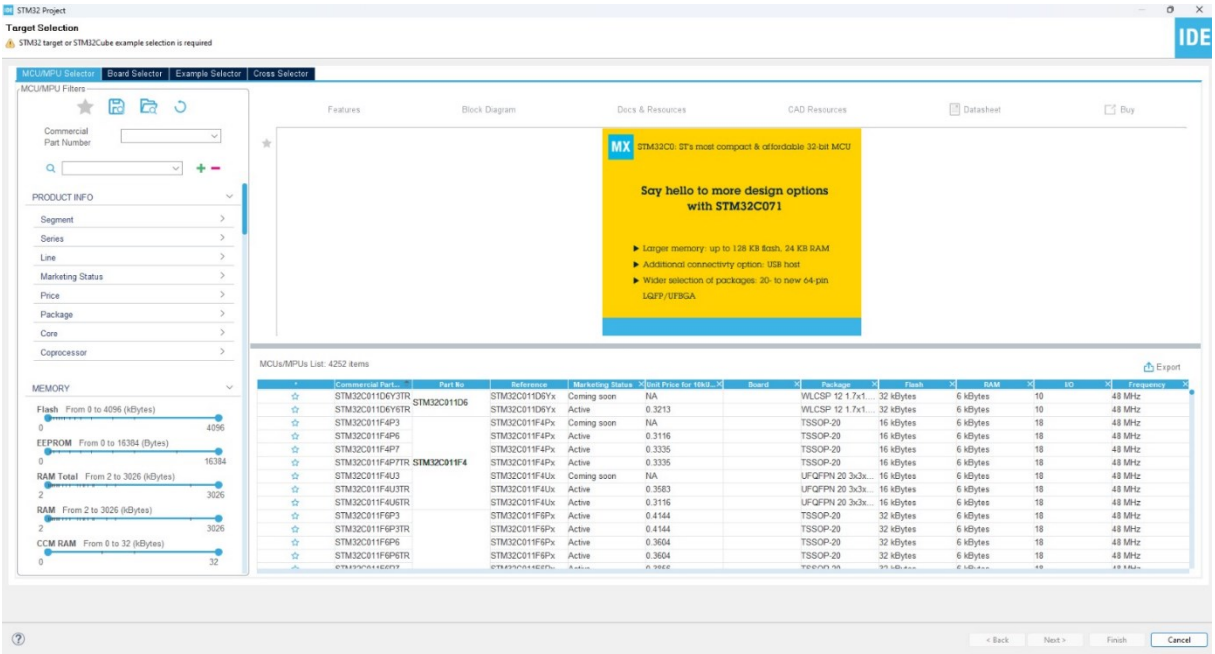
3.2.1 Step 1

Create a new project by selecting "STM32 Project" from the list.

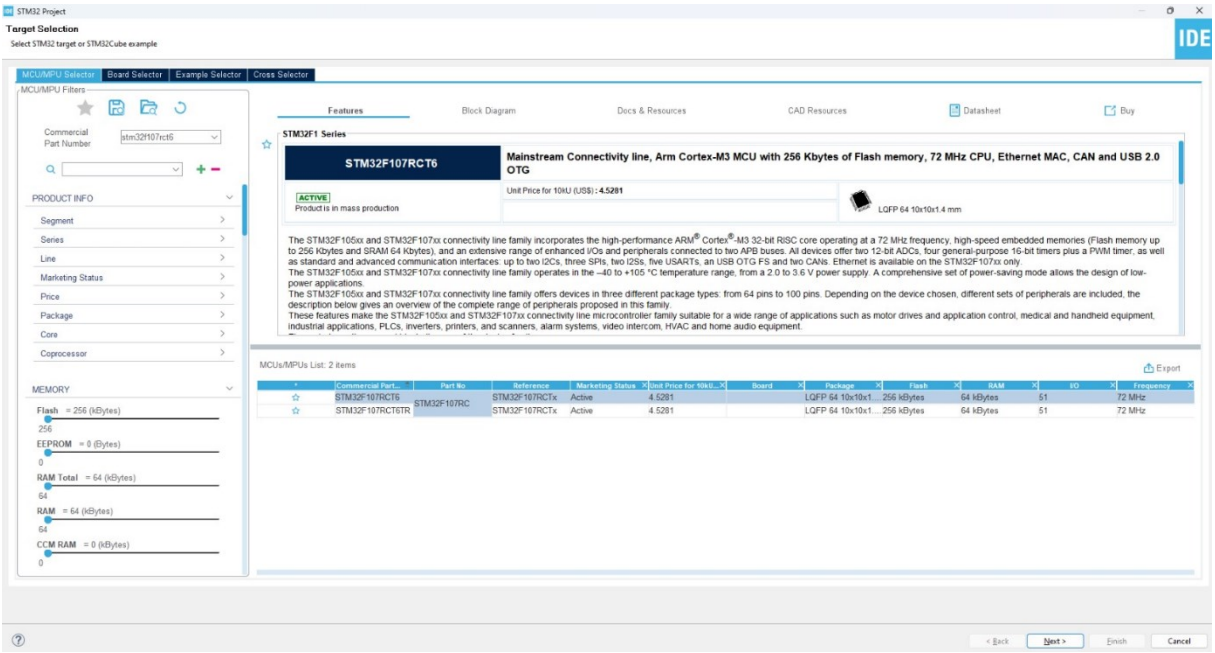


3.2.2 Step 2

After loading some information about the available microcontrollers online, the IDE will show up a new window, where you can select the specific microcontroller you are working with:

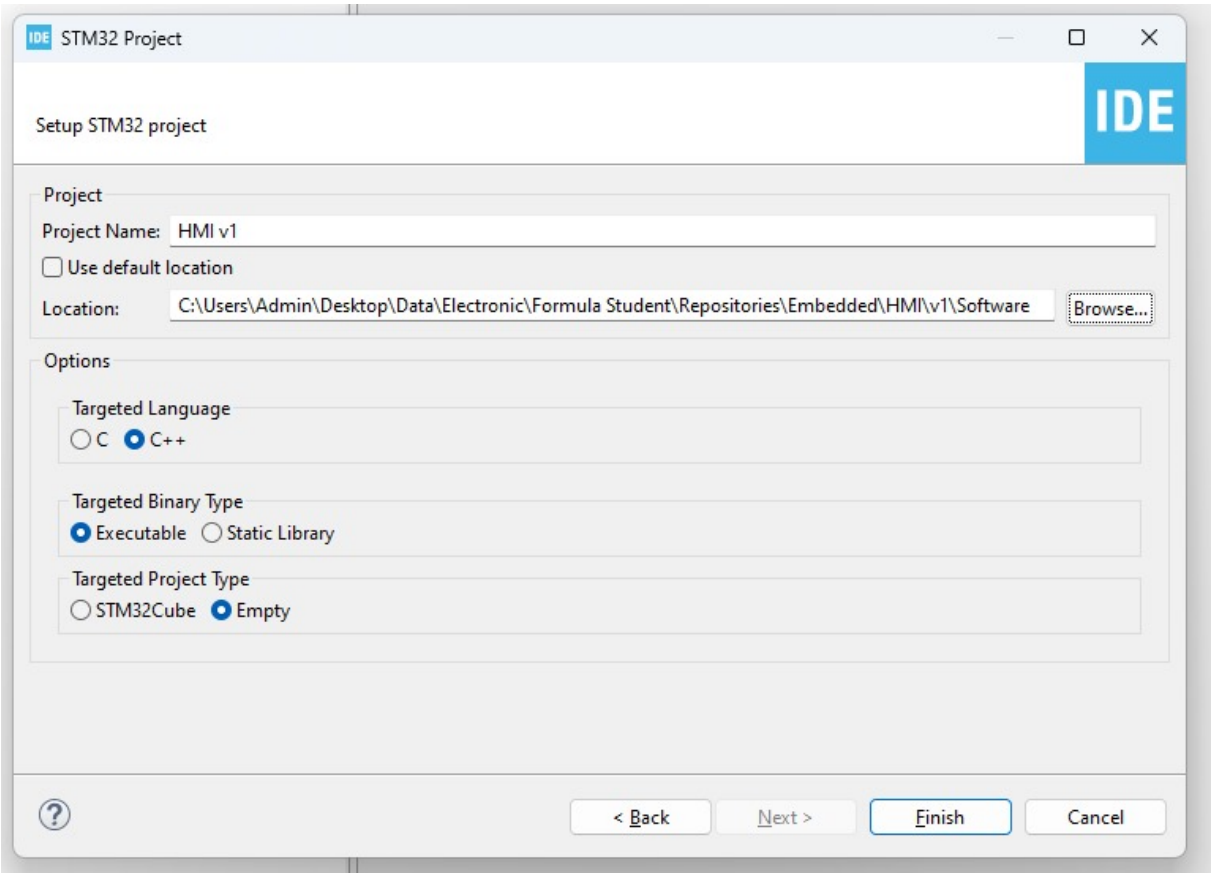


In the "Commercial Part Number" field, enter your microcontroller's name (in this example we take the STM32F107RCT6) and select the correct microcontroller from the bottom window. Select "Next" on the bottom to get to the next window.



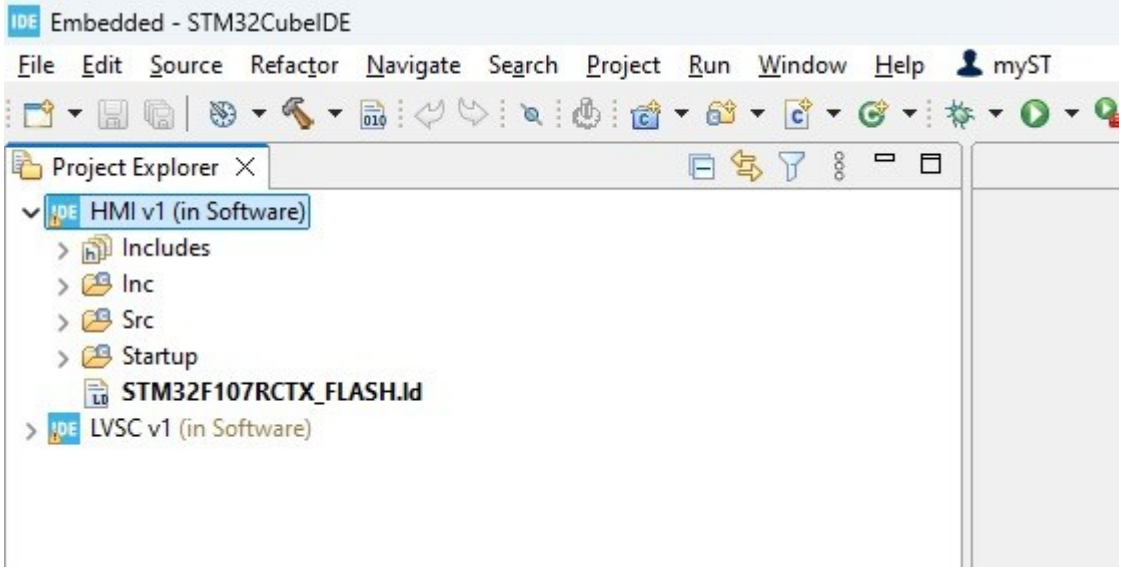
3.2.3 Step 3

Configure the settings as shown in the image below. The path and project name must be set specific to your project. The project location can be left at "Use default location". Remember that the project location selected must be empty.

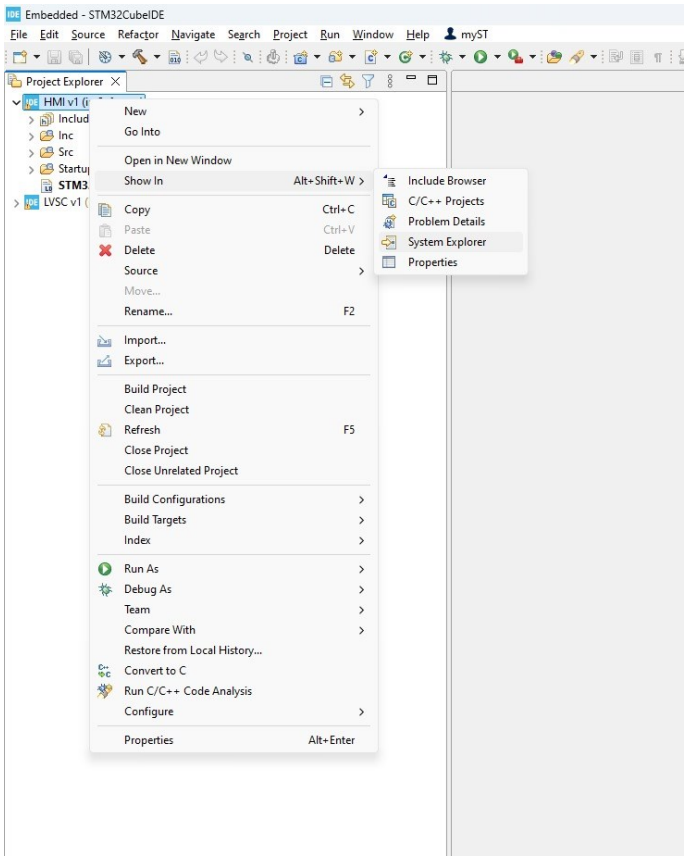


3.2.4 Step 4

After hitting the "Finish" button, the project explorer should show your freshly created project (the "LVSC v1" project here shouldn't be there on your fresh installation).



Right-click the project name and select "Show in"->"System Explorer"



Enter the folder ("Software" in this example) and delete

- All files with extension ".ld"

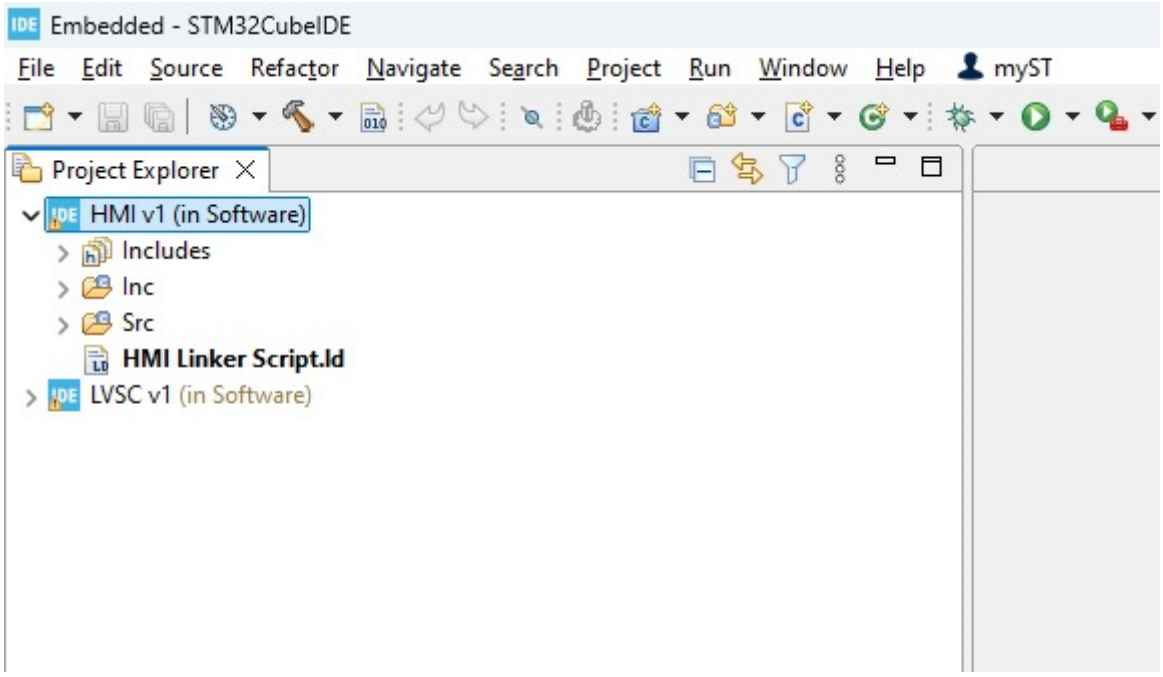
- The "Startup" folder

- All files inside the "Inc" and "Src" folders

Now copy the Linker Script (create your own or take one from the examples) into the project directory.

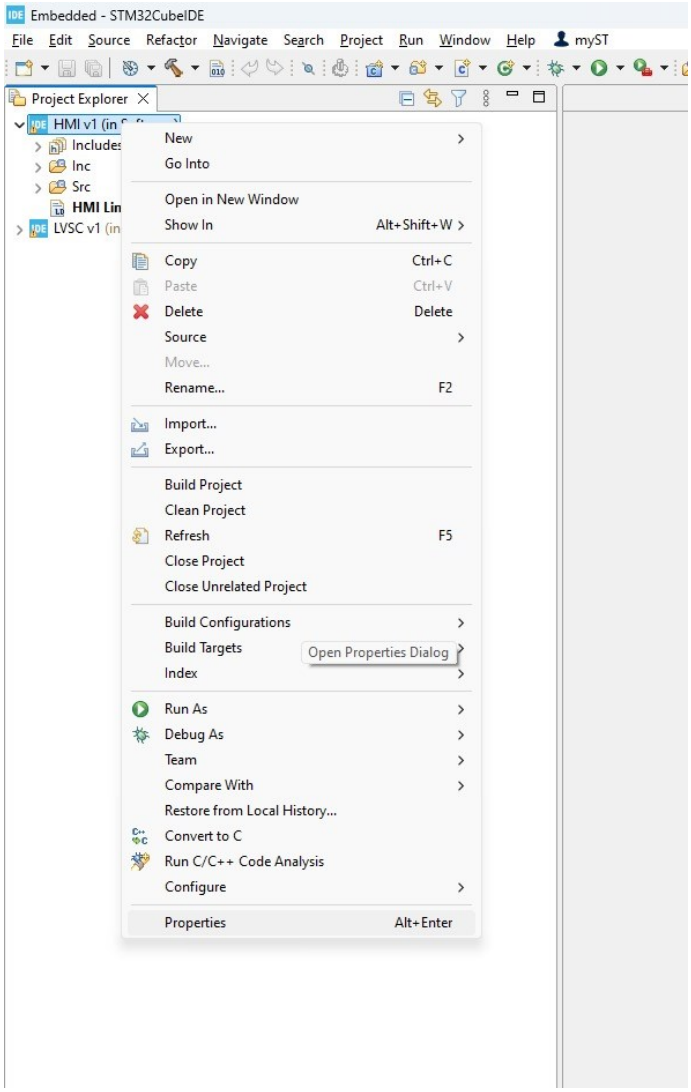
Also copy the standard "main.cpp" and "main.hpp" files (your own or taken from the examples) into the "Src" folder (for .cpp files) or the "Inc" folder (for .hpp files).

To update the project explorer in the IDE, select the project by clicking on the project name and hit "F5". It should look like this (without the "LVSC v1" project again).

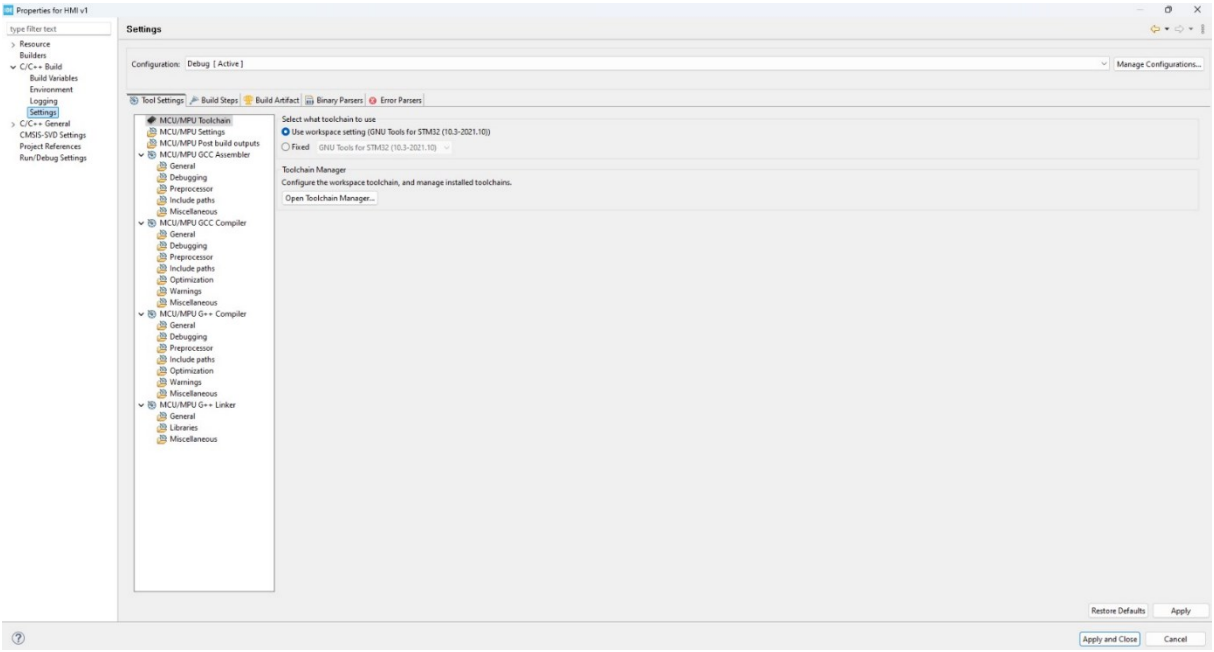


3.2.5 Step 5

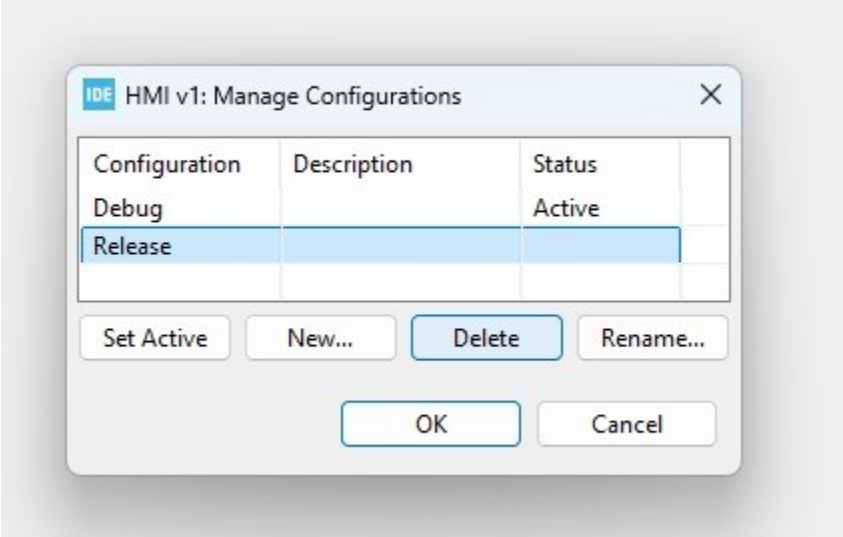
Now it's time to make the project-specific settings. They are the same for nearly all microcontrollers with a processor other than the Cortex M7. Open the settings by right-clicking on the project name and selecting "Properties".



Navigate to "C/C++ Build"->"Settings"

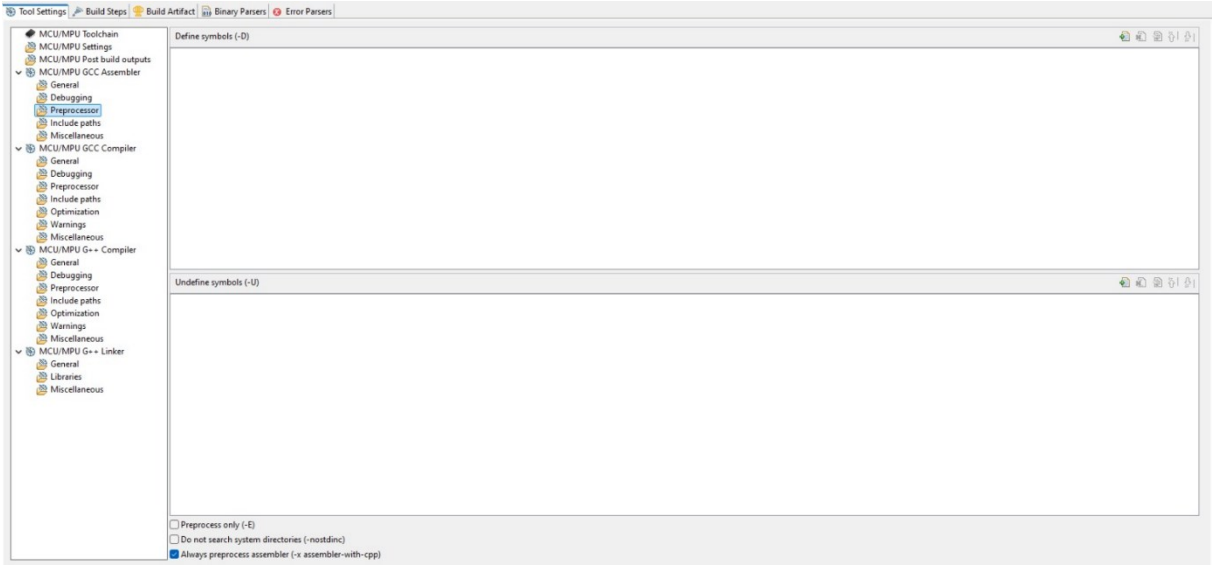


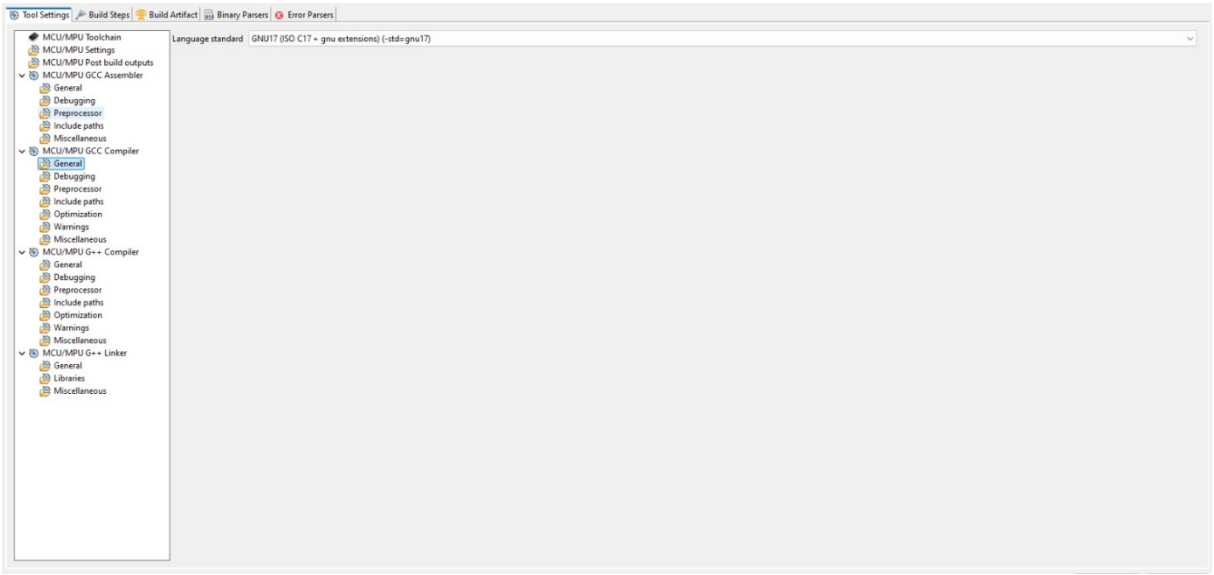
Hit the "Manage Configurations" button on the top right and delete the “Release” configuration by selecting it and clicking "Delete" in the little window that popped up. Hit "OK" then.

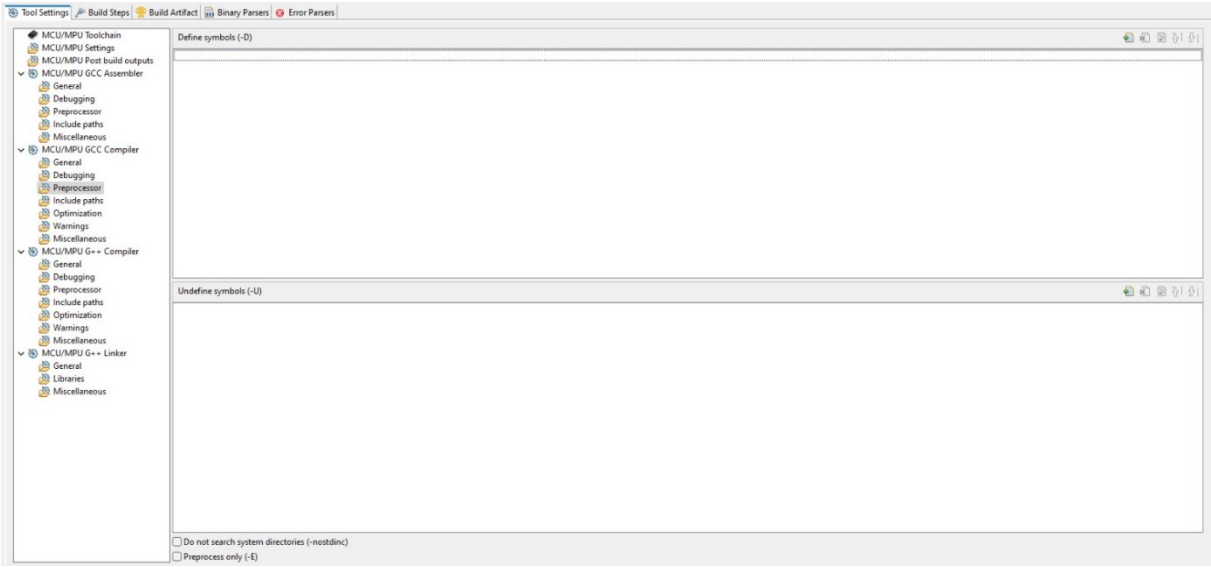


3.2.6 Step 6

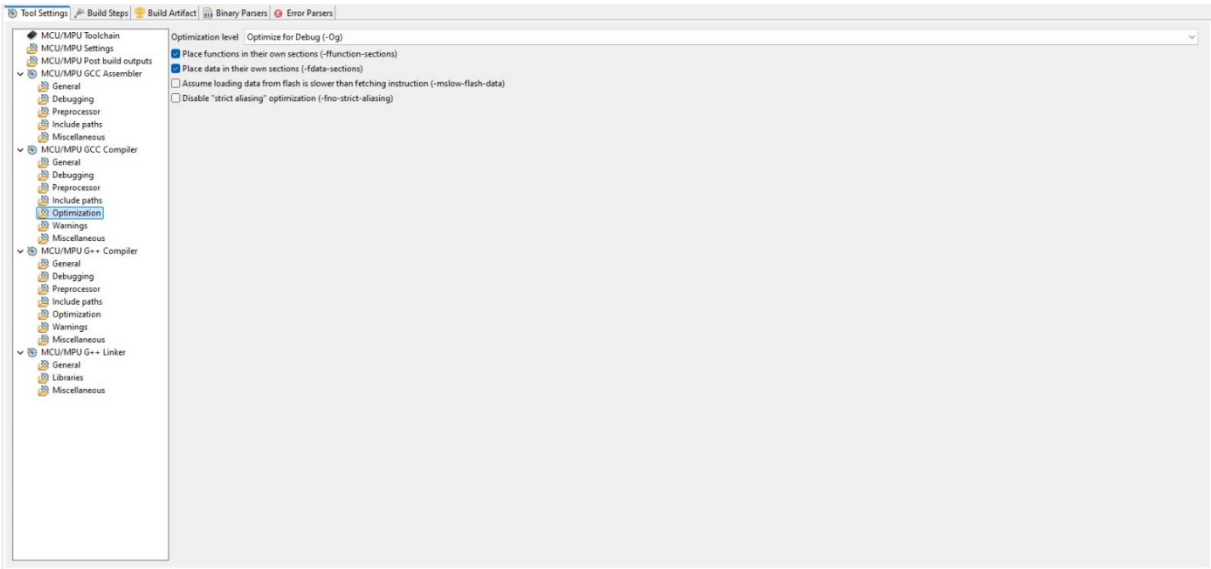
Now, simply change all settings so that they look like in the images below...

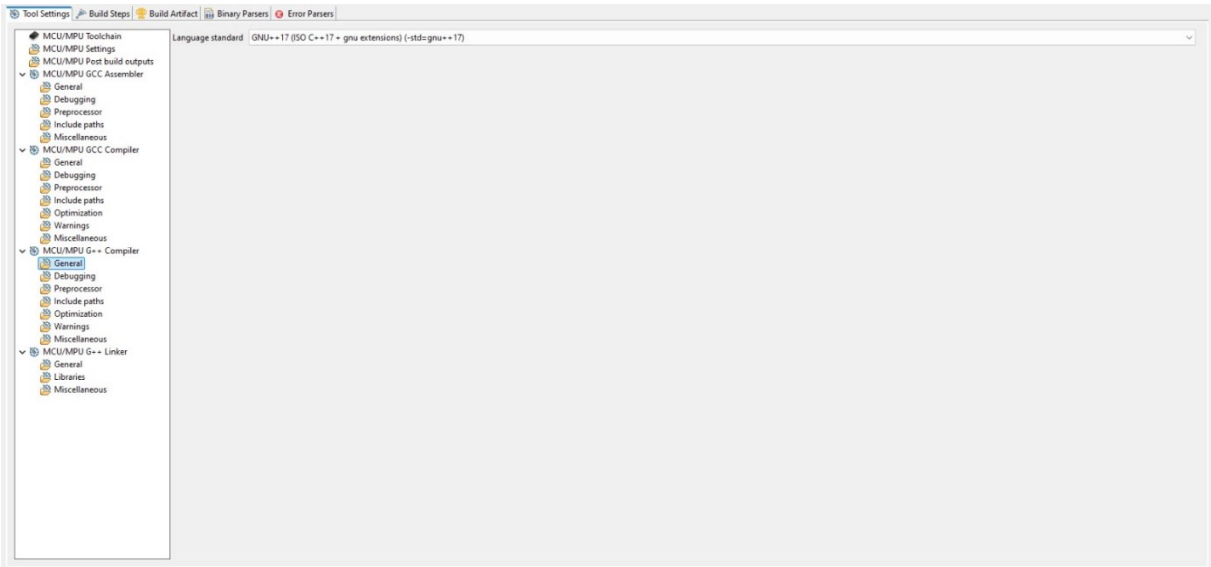





Tool Settings
Build Steps
Build Artifact
Binary Parsers
Error Parsers
MCU/MPU Toolchain
MCU/MPU Settings
MCU/MPU Post build outputs
MCU/MPU GCC Assembler
General
Debugging
Preprocessor
Include paths
Miscellaneous
MCU/MPU GCC Compiler
General
Debugging
Preprocessor
Include paths
Optimization
Warnings
Miscellaneous
MCU/MPU G++ Compiler
General
Debugging
Preprocessor
Include paths
Optimization
Warnings
Miscellaneous
MCU/MPU G++ Linker
General
Libraries
Miscellaneous
Define symbols (-D)
Undefine symbols (-U)
Do not search system directories (-nostdinc)
Preprocess only (-E)

Tool Settings
Build Steps
Build Artifact
Binary Parsers
Error Parsers
MCU/MPU Toolchain
MCU/MPU Settings
MCU/MPU Post build outputs
MCU/MPU GCC Assembler
General
Debugging
Preprocessor
Include paths
Miscellaneous
MCU/MPU GCC Compiler
General
Debugging
Preprocessor
Include paths
Optimization
Warnings
Miscellaneous
MCU/MPU G++ Compiler
General
Debugging
Preprocessor
Include paths
Optimization
Warnings
Miscellaneous
MCU/MPU G++ Linker
General
Libraries
Miscellaneous
Include paths (-I)
Include files (-include)


Tool Settings
Build Steps
Build Artifact
Binary Parsers
Error Parsers
MCU/MPU Toolchain
MCU/MPU Settings
MCU/MPU Post build outputs
MCU/MPU GCC Assembler
General
Debugging
Preprocessor
Include paths
Miscellaneous
MCU/MPU GCC Compiler
General
Debugging
Preprocessor
Include paths
Optimization
Warnings
Miscellaneous
MCU/MPU G++ Compiler
General
Debugging
Preprocessor
Include paths
Optimization
Warnings
Miscellaneous
MCU/MPU G++ Linker
General
Libraries
Miscellaneous
Optimization level
Optimize for Debug (-Og)
Place functions in their own sections (-ffunction-sections)
Place data in their own sections (-fdata-sections)
Assume loading data from flash is slower than fetching instruction (-mslow-flash-data)
Disable "strict aliasing" optimization (-fno-strict-aliasing)

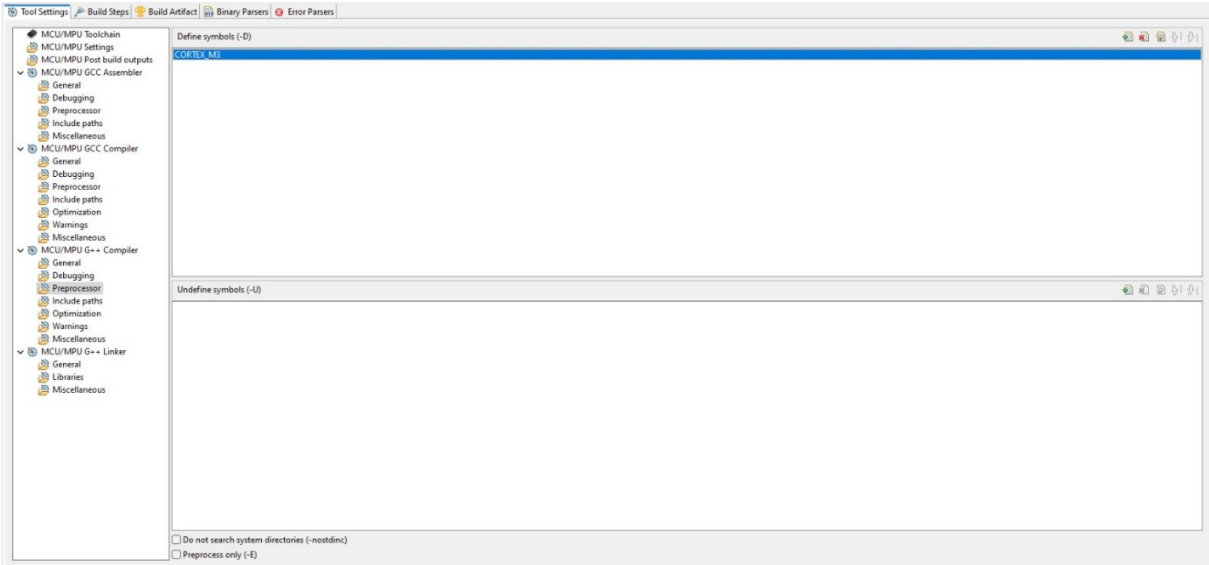

Tool Settings
Build Steps
Build Artifact
Binary Parsers
Error Parsers
MCU/MPU Toolchain
MCU/MPU Settings
MCU/MPU Post build outputs
MCU/MPU GCC Assembler
General
Debugging
Preprocessor
Include paths
Miscellaneous
MCU/MPU GCC Compiler
General
Debugging
Preprocessor
Include paths
Optimization
Warnings
Miscellaneous
MCU/MPU G++ Compiler
General
Debugging
Preprocessor
Include paths
Optimization
Warnings
Miscellaneous
MCU/MPU G++ Linker
General
Libraries
Miscellaneous
Language standard
GNU++17 (ISO C++17 + gnu extensions) (-std=gnu++17)

Here you have to enter the name of your processor. The processor type depends on the microcontroller you are using. Examples:

- Cortex M0 will be "CORTEX_M0"

- Cortex M0+ will be "CORTEX_M0P"

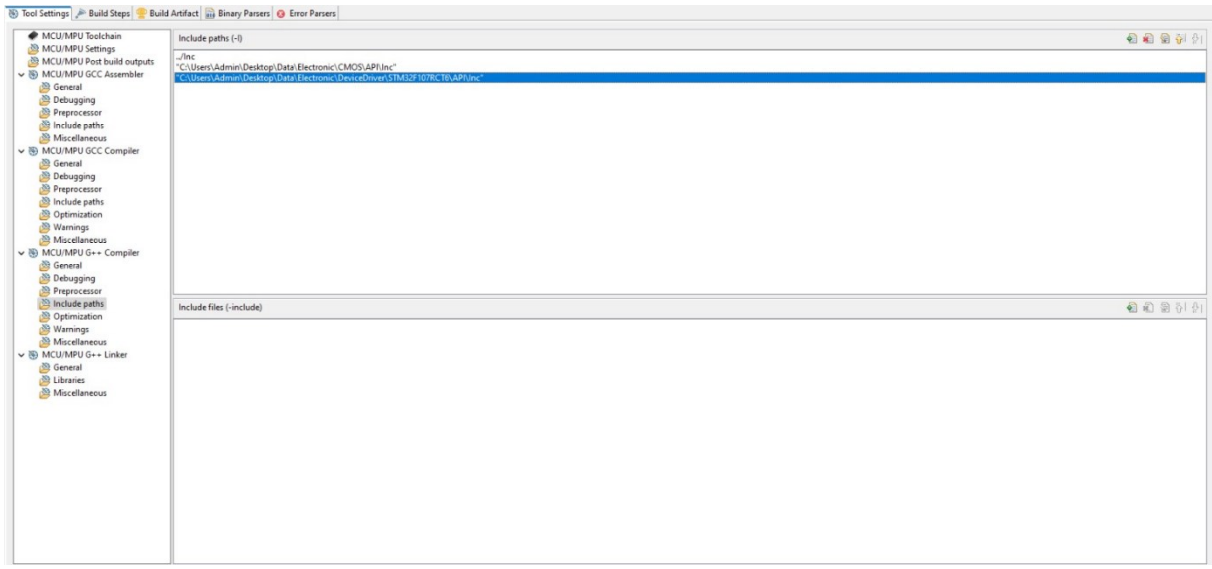
- Cortex M7 will be "CORTEX_M7"

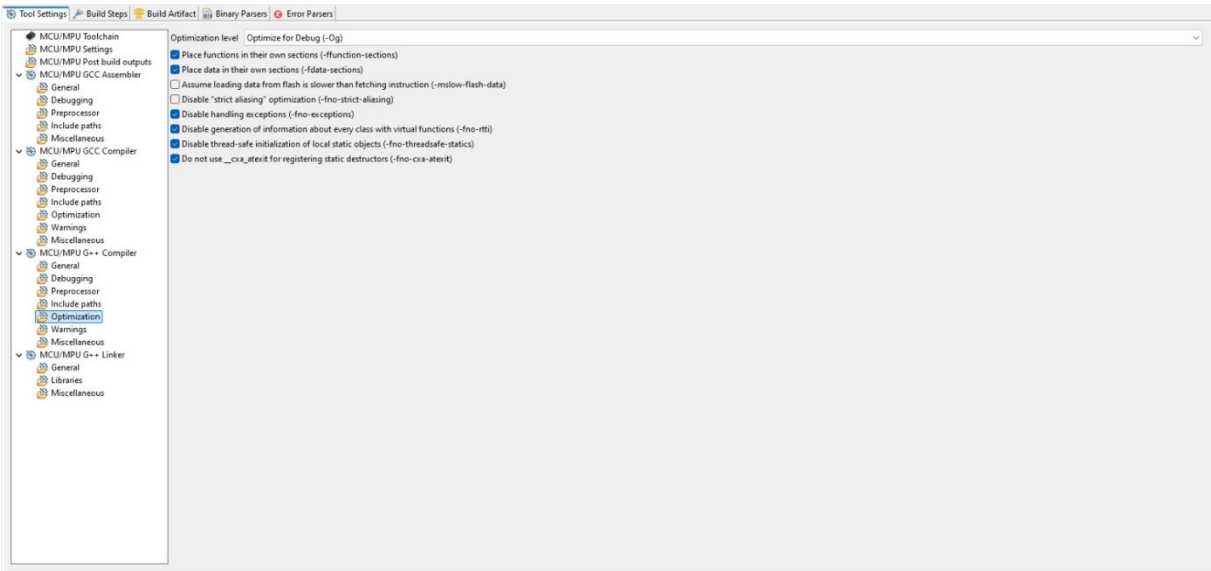


Important: The first path that you add to this list must be the path to the CMOS repository, the second path the path to your DeviceDriver.

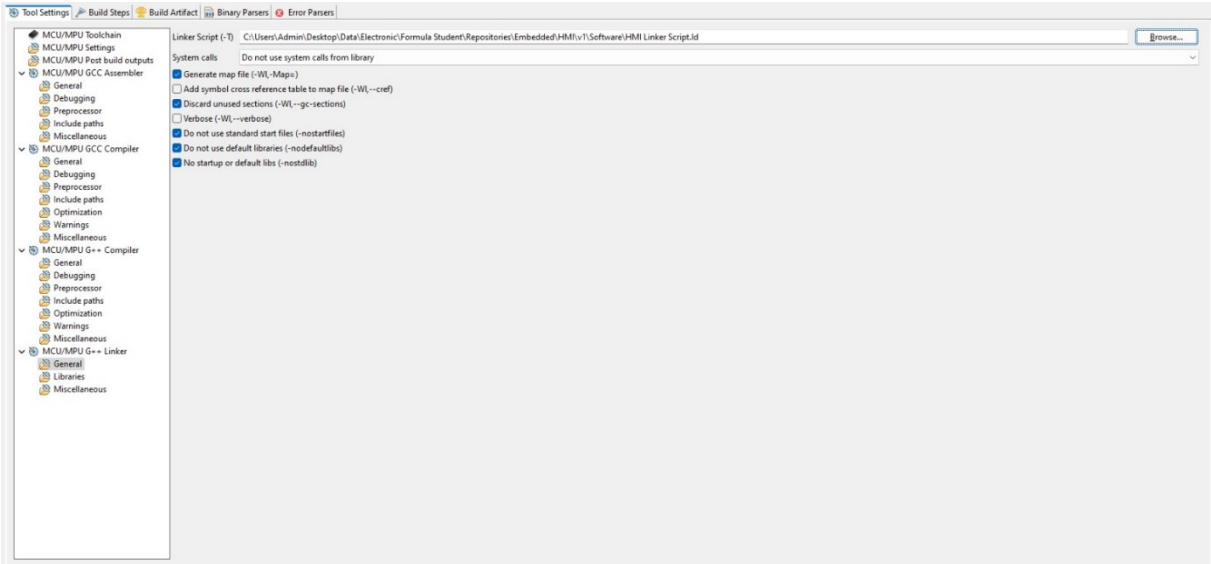
By clicking on "File System" you can browse your file system for these paths.

Choose the subfolder "API/Inc" here. In the DeviceDriver library, you first have to navigate to the subfolder of your specific microcontroller and then to the "API/Inc" subfolder.





To select the correct linker script, hit the "Browse" button and select the linker script mentioned in Step 4.



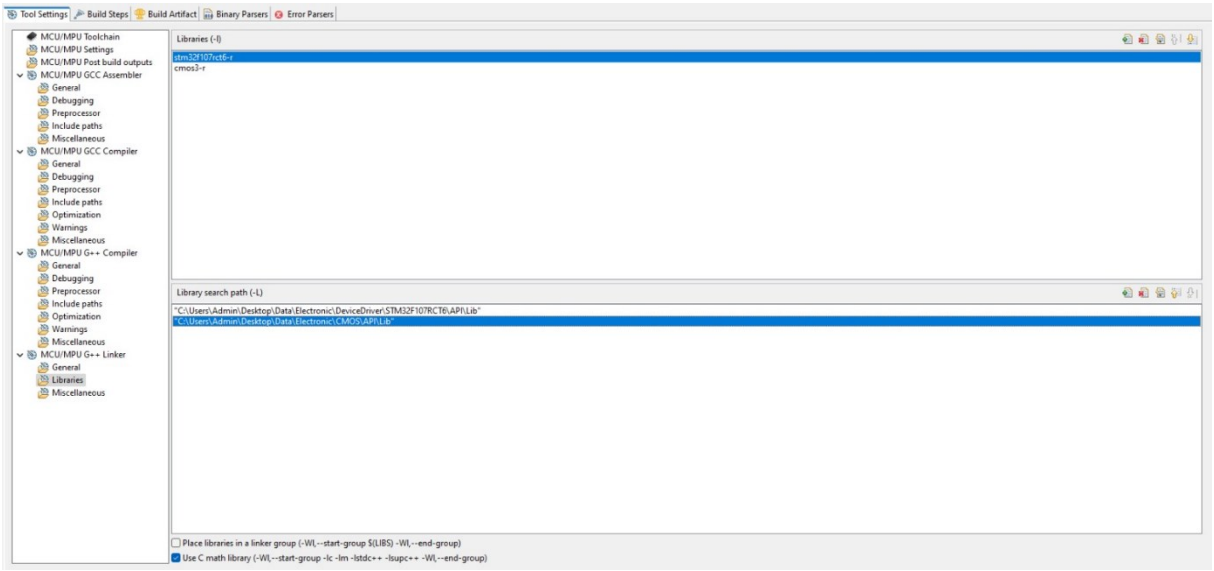
Important: This time the DeviceDriver library must be entered first!

Two libraries must be entered in the top window:

1. The DeviceDriver library: It’s your microcontroller's name in small letters with the suffix "-r"

2. The CMOS library compiled for your processor core: Its "cmos" with the number of your processor ("3" for Cortex M3, "0p" for Cortex M0+ and so on...) and again the suffix "-r".

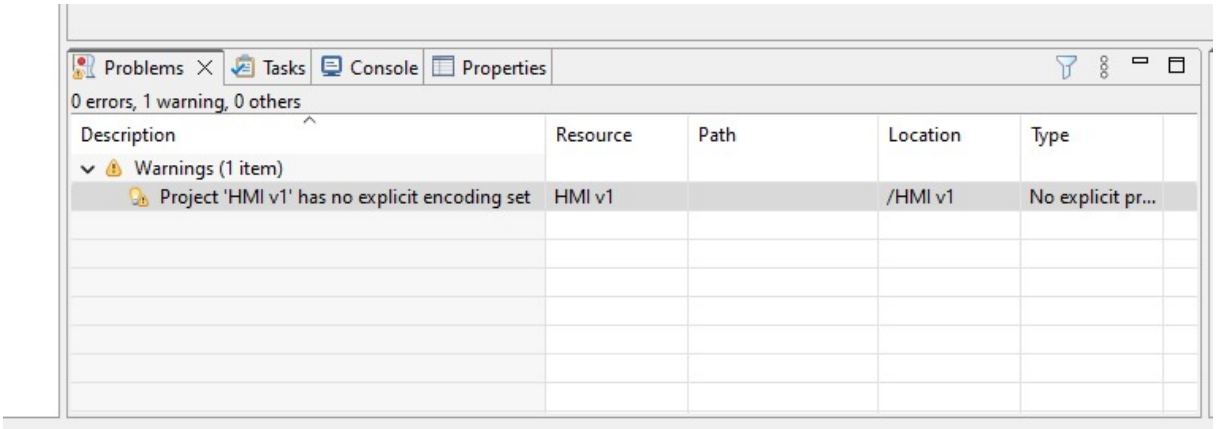
In the bottom window, you have to enter the paths to these libraries: They are the same paths as you have entered above already in the "MCU/MPU G++ Compiler"->"Include Paths" tab. But the subfolder now is not "Inc", but "Lib". This time the path to the DeviceDriver's library must be above the one to the CMOS repository.



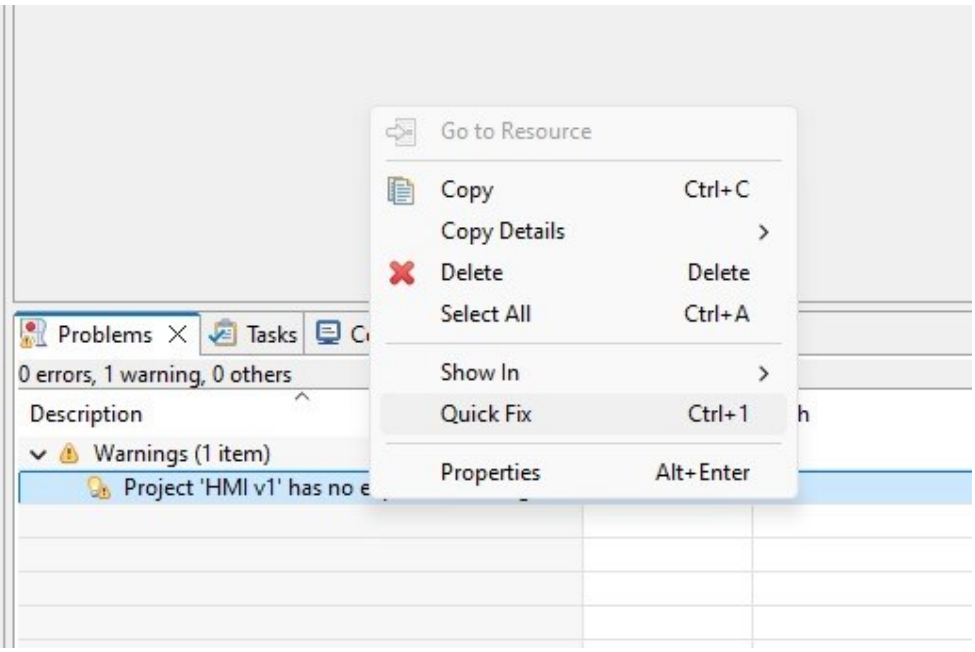
Now hit "Apply and Close" on the bottom right.

3.2.7 Step 7

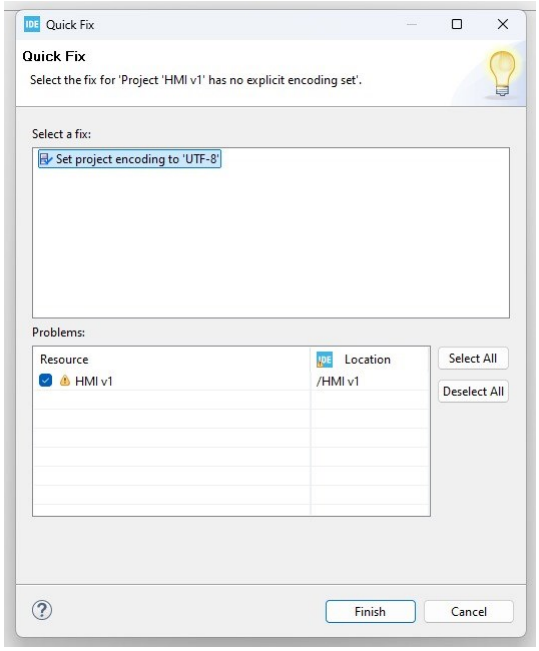
Now only one simple step is missing until the project is ready to be compiled: You may have noticed a little warning in the "Problems" tab on the bottom.



To get rid of this, simply right-click it and select "Quick Fix"

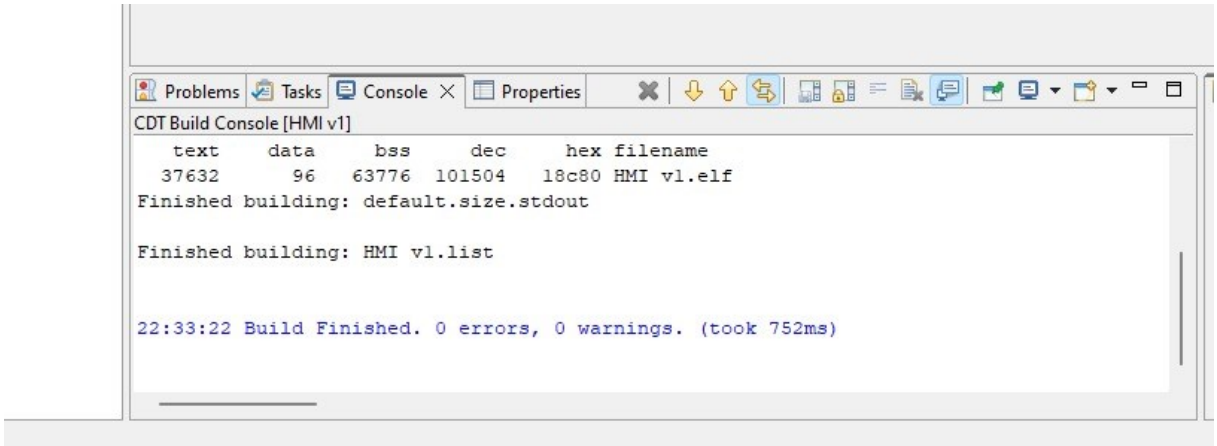


Select "Finish" and the warning should be gone.



3.2.8 Step 8

Now we can compile the project the first time. Hit "Ctrl + B" to compile - your console output on the bottom should look like this (the actual numbers there may differ).

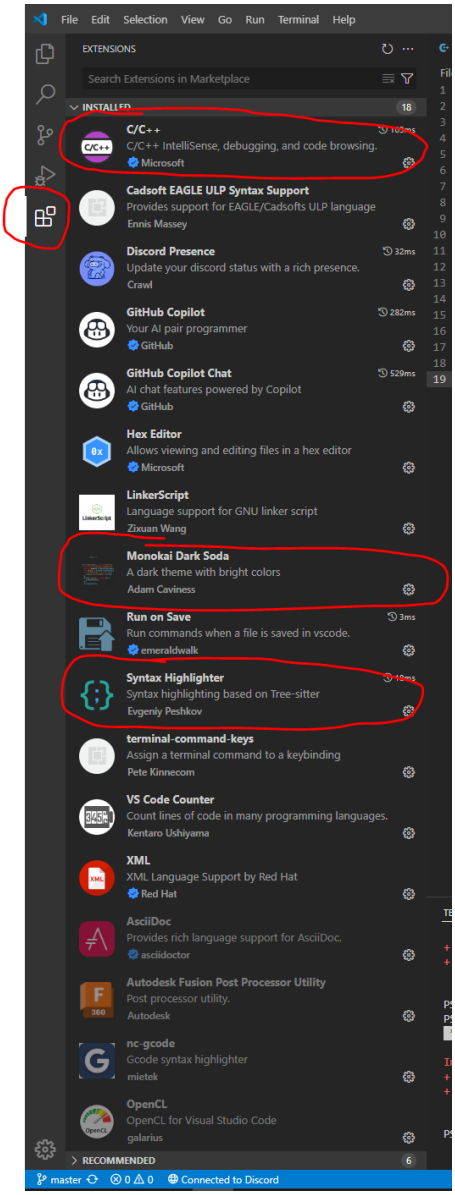


If there are some errors – contact me on Github.

4 VS Code

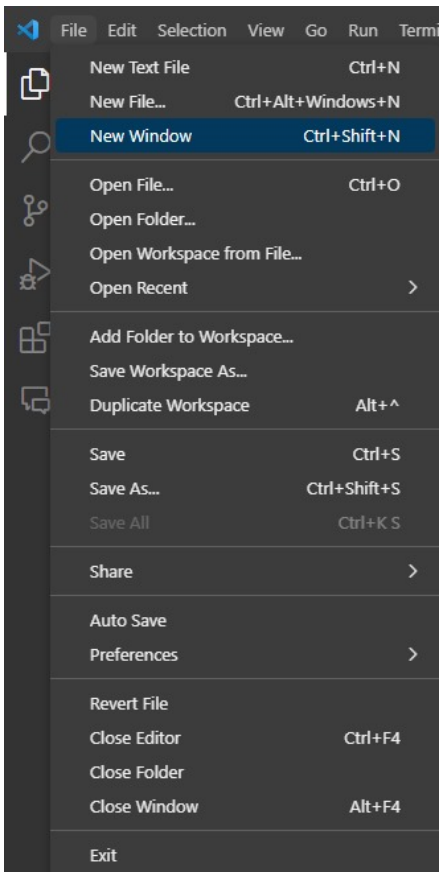
4.1 Extensions

Open the software and click on the "Extension" symbol on the left bar. Then install the marked extensions ("Monokai Dark Soda" is only a color theme and therefore optional).



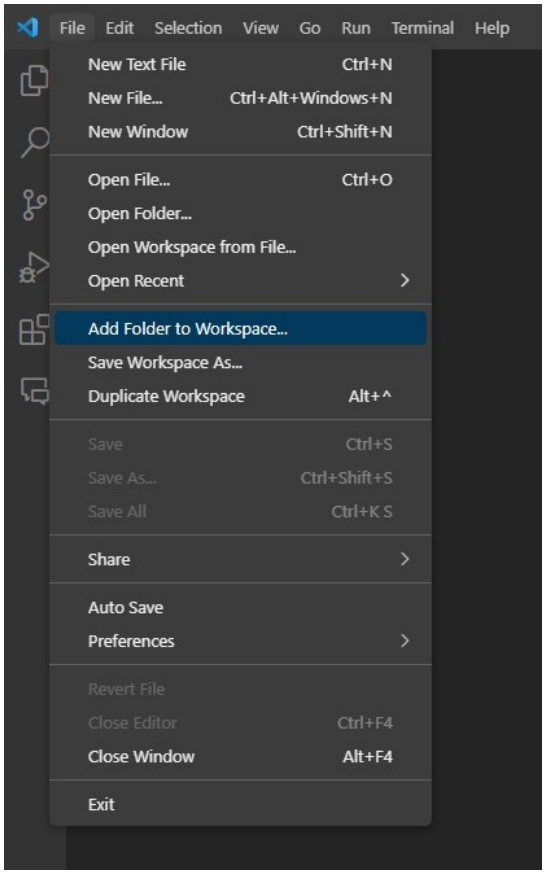
4.2 Workspace

Open a new instance of VS Code - if there are already some files or folders opened, go to "File"->"New Window" so that a fresh clean new window opens and close the old window.

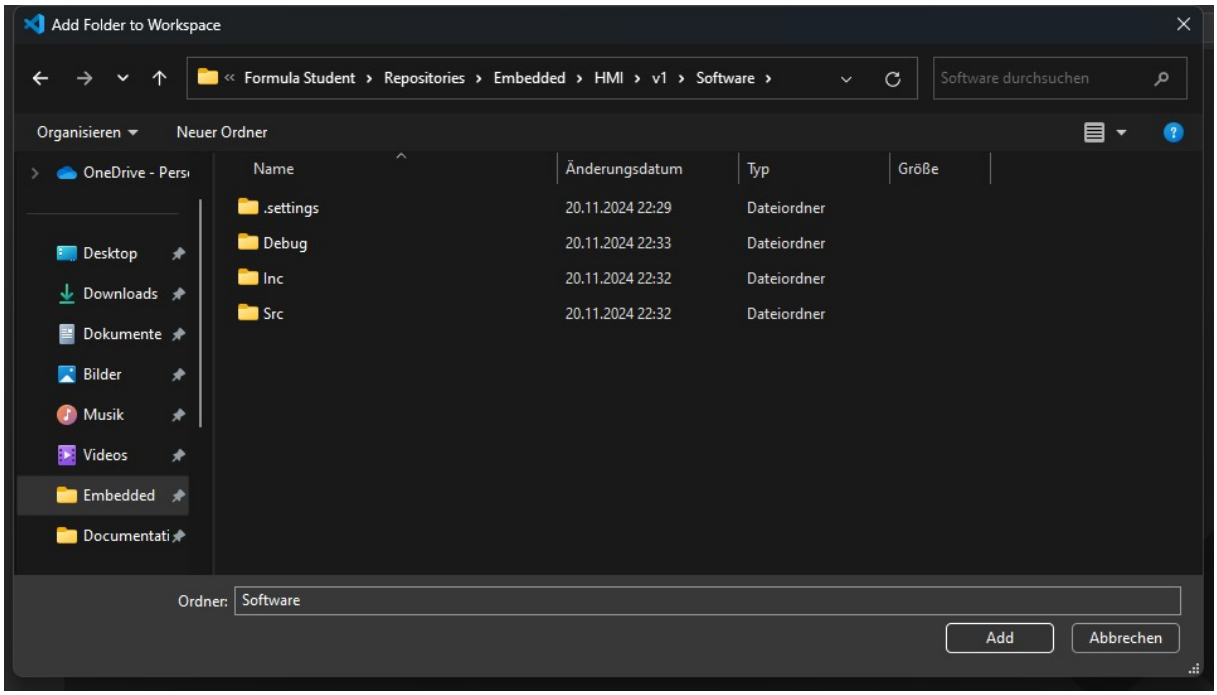


Now add the project folder ("HMI v1" in this example) by selecting

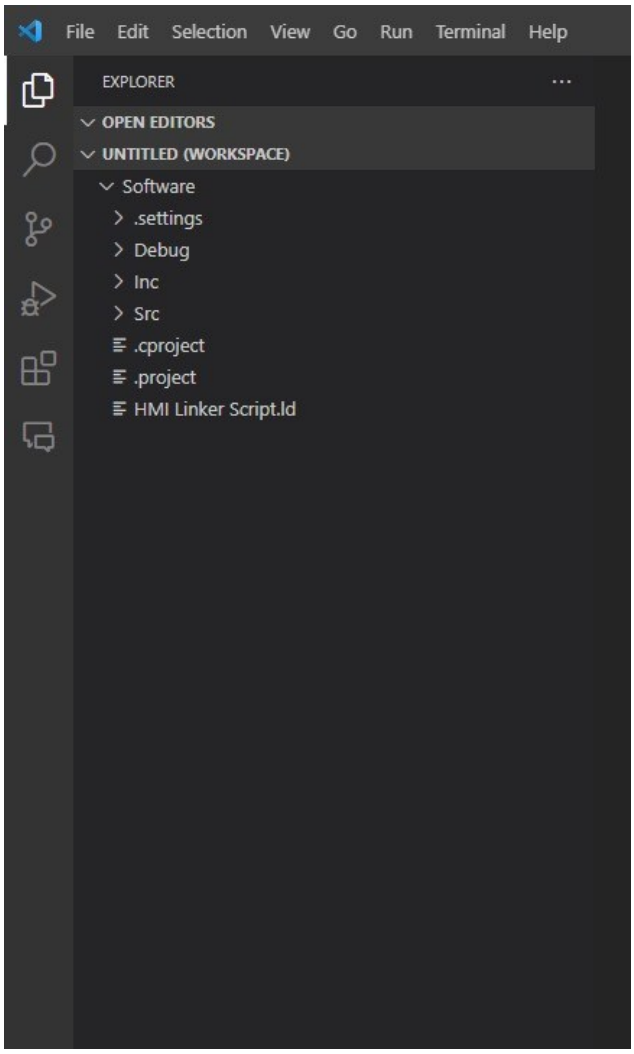
"File"->"Add Folder to Workspace"



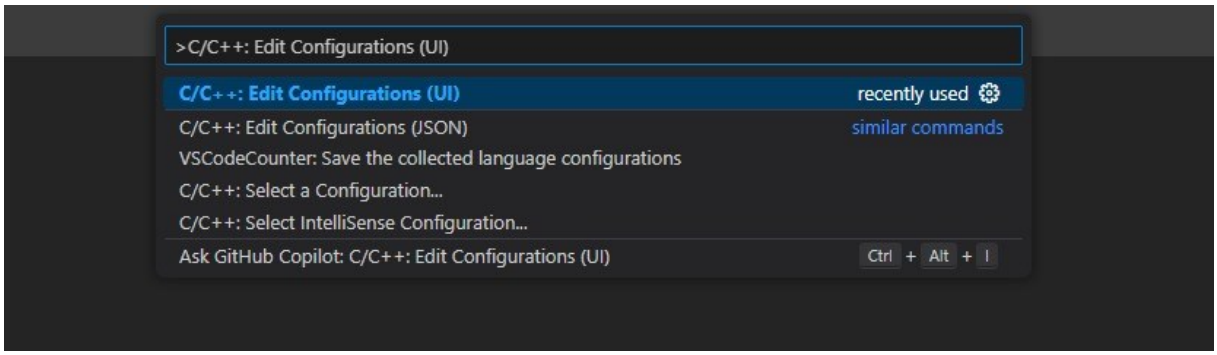
Select the folder where the project is located in and press "Add".



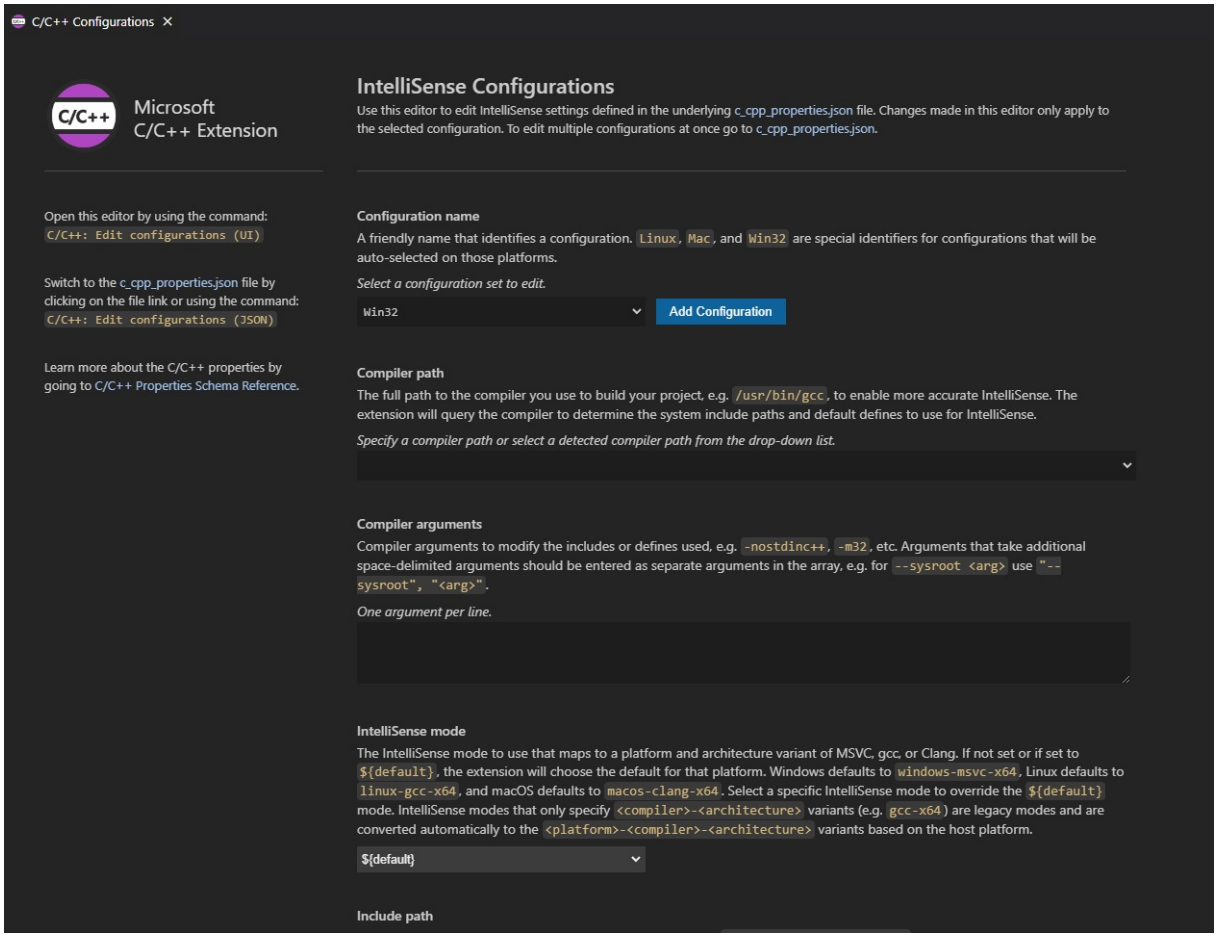
The project explorer in VS Code should now look like this:



Now hit "F1" and type "C/C++: Edit Configurations (UI)". Select this command with “Enter”.



The view should be like this:



For the configuration name enter:

- "ARMv6-M" if the processor is a Cortex M0 or Cortex M0+

- "ARMv7-M" if the processor is a Cortex M1, Cortex M3, Cortex M4 or Cortex M7

- "ARMv8-M" if the processor is a Cortex M23, Cortex M33 or a Cortex M35P

- "ARMv8.1-M" if the processor is a Cortex M52, Cortex M55 or a Cortex M85

The compiler path depends on your installation of the STM32CubeIDE. To find it, navigate to "C:/ST", then the subfolder of the version of your installation (something like "STM32CubeIDE_1.16.0"), then "STM32CubeIDE/plugins".

Now you need to find the folder that starts with:

"com.st.stm32cube.ide.mcu.externaltools.gnu-tools-for-stm32.10.3-2021.10.win32_" and then ends in some cryptic numbers.

Inside this folder, navigate to "tools/bin".

Copy the path to this location now from the windows explorer and insert it into the compiler path in VS Code. Append a "\arm-none-eabi-g++" and we are fine.

On inserting it, there may pop up a little console on the bottom of VS Code, telling you something like "Unable to resolve Configuration with compilerPath: ..." - just close it.

Now configure the IntelliSense Mode to "windows-gcc-arm".

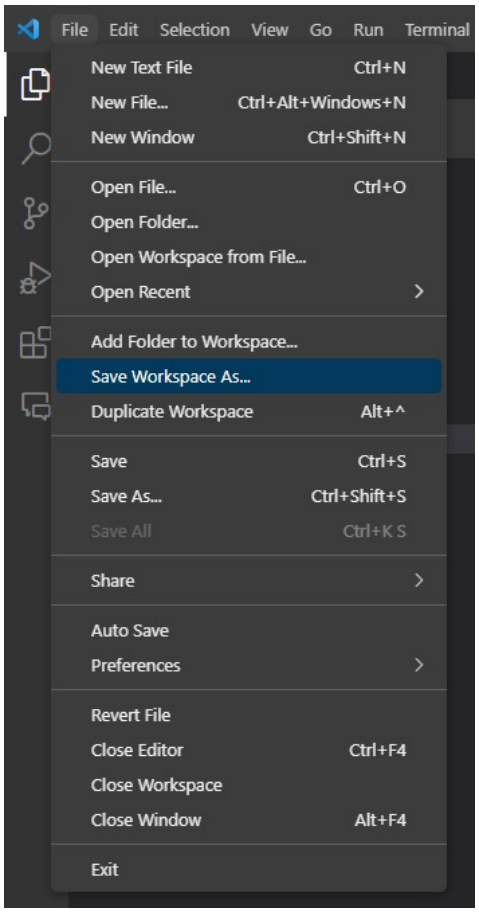
For the “Include Paths”, add (every path in a new line) the same paths that you have added in the STM32CubeIDE at Step 6 in the "MCU/MPU G++ Compiler"->"Include Paths" tab.

Add the suffix "/`**`" to these paths.

In the “Defines” section, delete all existing symbols and then define the same symbol that you have already defined in the STM32CubeIDE at Step 6 in the "MCU/MPU G++ Compiler"->"Preprocessor" tab.

Set the C Standard to "gnu17" and the C++ Standard to "gnu++17".

Finally, close the "C/C++ Configurations" file and select "File"->"Save Workspace as..." and save this workspace configuration to the project location. You can re-open this workspace simply by double-clicking this file or by using "File"->"Open Workspace from File" or by selecting it from the "Recent"-List with "File"->"Open Recent".



5 Revision History

Revision	Date	Changes
0	03/2026	Document created